CONECTAR FIREBASE A TU APK ANDROID

API REST de Productos con Realtime Database

Brayan Quispe Mamani

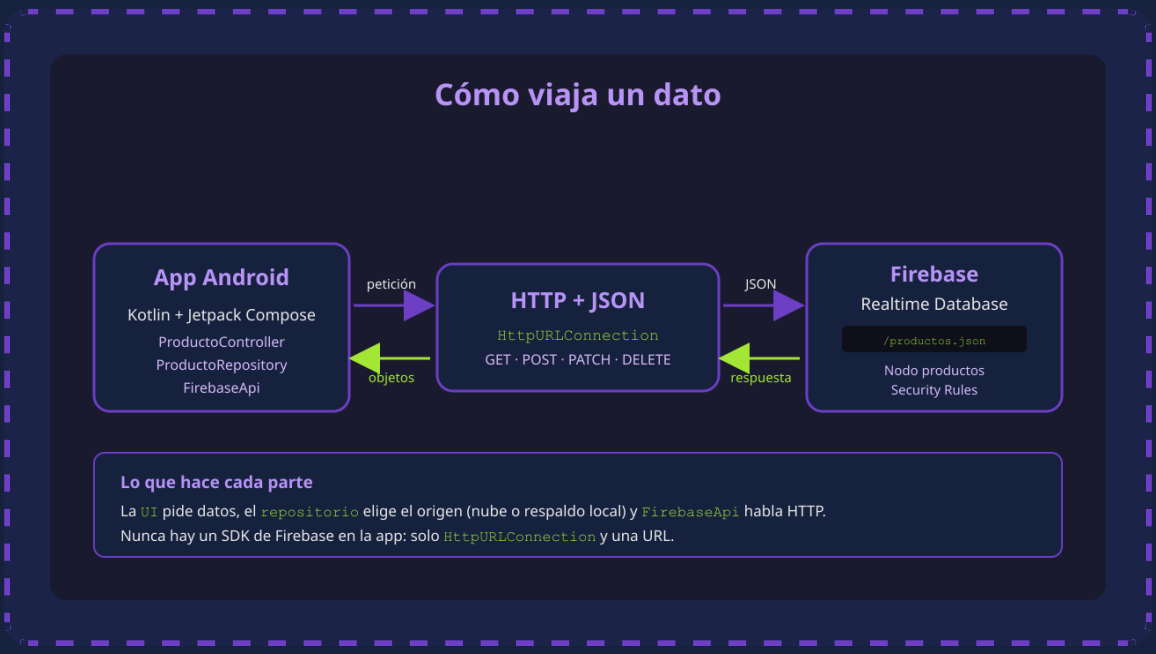
Programación de Dispositivos Móviles

Universidad Franz Tamayo — 2026

La Idea en 30 Segundos

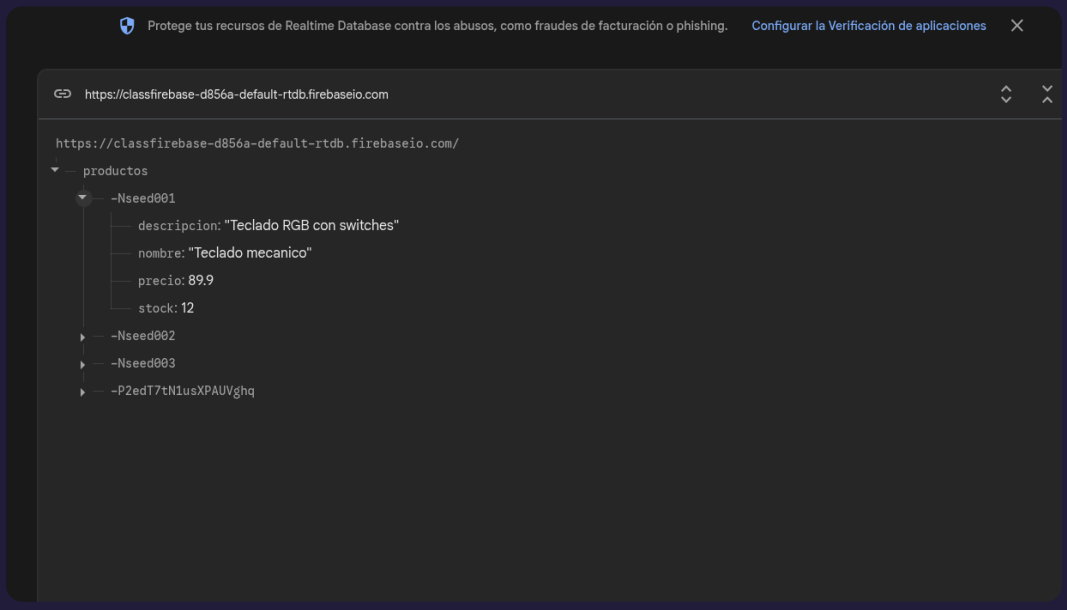
Tu app Android **no se conecta directo** a Firebase con un SDK pesado. En su lugar:

1. Firebase Realtime Database **expone una URL REST**.
2. Tu app hace peticiones **HTTP** (GET, POST, PATCH, DELETE).
3. Los datos viajan en **JSON**.
4. Vos controlás todo con `HttpURLConnection` y corrutinas.



1 Crear el proyecto en Firebase

- Entrá a **console.firebase.google.com** y elegí *Agregar proyecto*.
- Ponele un nombre: `classfirebase-d856a`.
- En el menú lateral: **Realtime Database** y luego *Crear base de datos*.
- Elegí **modo de prueba** (para desarrollo).
- Copiá la URL que aparece arriba: `https://classfirebase-d856a-default-rtdb.firebaseio.com`



> Esa URL es la dirección de la base de datos. Todo lo demás sale de acá.

2 Guardar la URL en el proyecto Android

En `local.properties` (nunca en el código):

```
firebase.db.url=https://classfirebase-d856a-default-rtdb.firebaseio.com
```

En `app/build.gradle.kts` del módulo:

```
buildConfigField("String", "FIREBASE_DB_URL", "\"${config("firebase.db.url")}\"")
```

Y activá `buildFeatures { buildConfig = true }`.





> Si cambiás de proyecto Firebase, editás una sola línea. El código no se toca.

3 Definir el modelo y las operaciones

Operación	Método HTTP	URL
Listar	GET	`/productos.json`
Crear	POST	`/productos.json`
Editar	PATCH	`/productos/<id>.json`
Eliminar	DELETE	`/productos/<id>.json`

El `.json` al final es **obligatorio** en Firebase REST.

La URL se arma en código, no se escribe a mano:

```
private fun rutaRaiz() = "$urlBase/$COLECCION.json"
```

```
2 Usages
private fun rutaRaiz() = "$urlBase/$COLECCION.json$autenticacion"

3 Usages
private fun ruta(id: String) = "$urlBase/$COLECCION/$id.json$autenticacion"

2 Usages
private val autenticacion: String
    get() = if (token.isBlank()) "" else "?auth=$token"
```

4 Hacer la petición desde Kotlin

Dentro de una función que se ejecuta en un hilo secundario:

```
val conexion = (URL(url).openConnection() as
HttpURLConnection).apply {
    requestMethod = metodo
    connectTimeout = TIEMPO_LIMITE_MS
}
```

Esa respuesta es un **JSON** que convertís a tus objetos `Producto`.

```
5 Usages
private fun peticion(metodo: String, url: String, cuerpo: String?): String {
    if (!configurada) {
        throw IllegalStateException(motivoSinConfigurar)
    }
    val conexion = (URL( spec = url).openConnection() as HttpURLConnection).apply {
        requestMethod = metodo
        connectTimeout = TIEMPO_LIMITE_MS
        readTimeout = TIEMPO_LIMITE_MS
        setRequestProperty("Accept", "application/json")
        if (cuerpo != null) {
            doOutput = true
            setRequestProperty("Content-Type", "application/json; charset=utf-8")
        }
    }
    try {
```

5 Cuidado con el hilo principal

- Las peticiones de red **no pueden ir en el hilo principal**.
- El repositorio envuelve todo el acceso en `withContext(Dispatchers.IO)`.
- La interfaz lanza la operación con `alcance.launch { ... }` y actualiza el estado al terminar.



> Acá NO se usó ViewModel: el estado se maneja en un controlador propio para este proyecto.

Errores que te van a aparecer

Código	Qué significa	Solución
404	La base de datos no existe	Creala en la consola Firebase
401/403	Reglas de seguridad bloquean	Ajustá las Security Rules
400	JSON mal formado	Revisá el body que enviás

El error más común: **olvidarse el** `.json` al final de la URL.

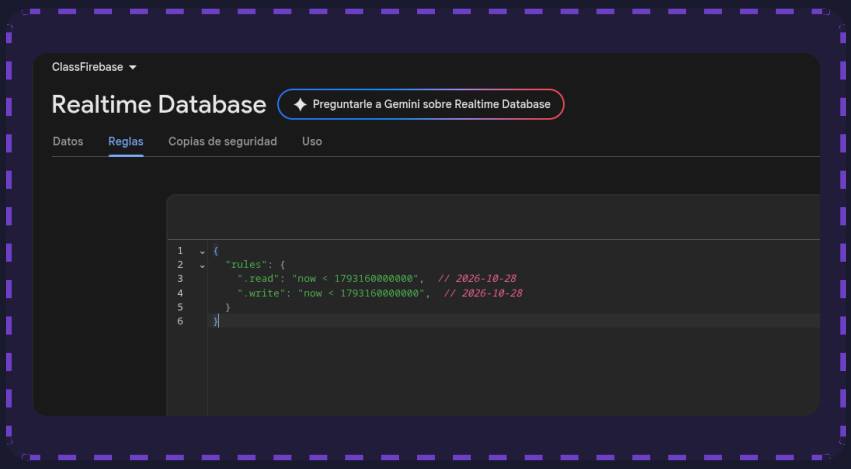
```
1 Usage
private fun explicar(codigo: Int, detalle: String): String = when (codigo) {
    401, 403 ->
        "Firebase rechazo la peticion (HTTP $codigo). Las reglas de seguridad bloquean el acceso: " +
            "en la consola abre la base de datos y ponla en Modo de prueba."
    404 ->
        "La base de datos no existe todavia (HTTP 404). Creala en la consola de Firebase: " +
            "Build > Realtime Database > Crear base de datos. URL usada: $urlBase"
    else -> "Firebase respondio HTTP $codigo: ${detalle.take( n = 140)}"
}
```

Seguridad: lo que NO hay que hacer

- No uses *database secrets*: **Firebase las eliminó en 2024**.
- No pongas la URL hardcodeada en el código.
- No dejes la base en modo prueba en producción.

Lo correcto

- Configurar **Security Rules** por nodo y por usuario.
- Guardar la URL en `local.properties`.
- Usar **Firebase Auth** si necesitás usuarios.



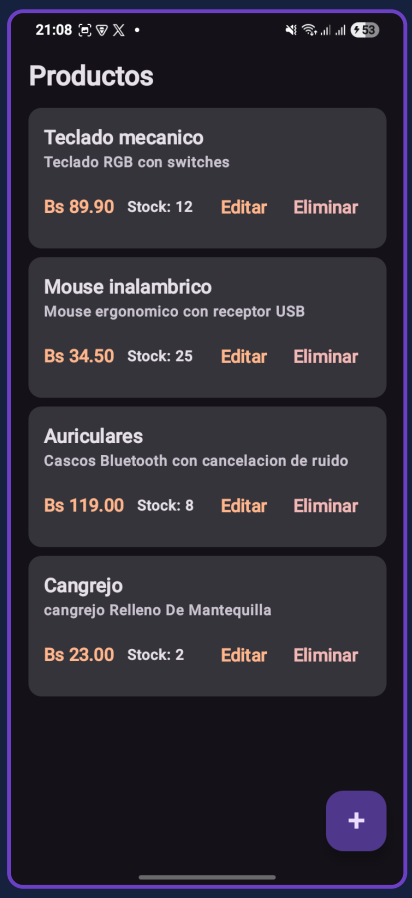
Lo que dice mi regla actual

`".read": "now < 1793160000000"` `".write": "now < 1793160000000"`

Es una bomba de tiempo. Ese número es una marca de tiempo Unix: el **28/10/2026**. Ese día `now` deja de ser menor, la regla da `false` y la app deja de leer y escribir **sin tocar una línea de código**. En producción se reemplaza por reglas con `auth`.

Resultado Final

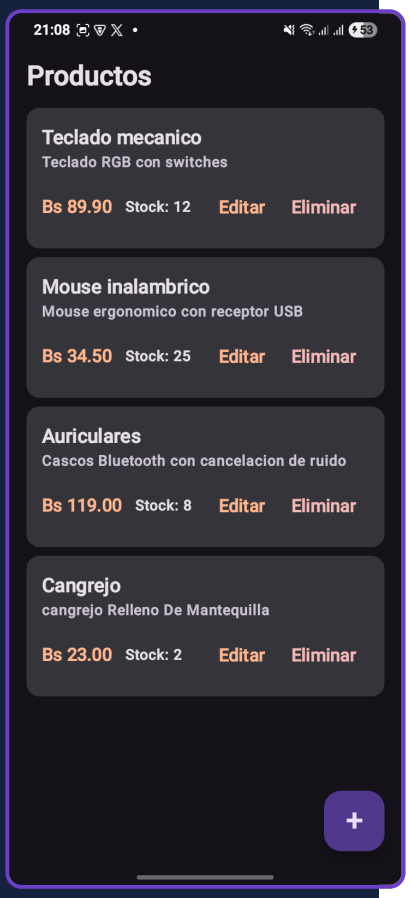
- App Android que **lista, crea, edita y elimina** productos.
- Datos sincronizados con **Firebase Realtime Database**.
- Respaldo local en **JSON** por si falla la red.
- Los 5 endpoints responden **HTTP 200**.



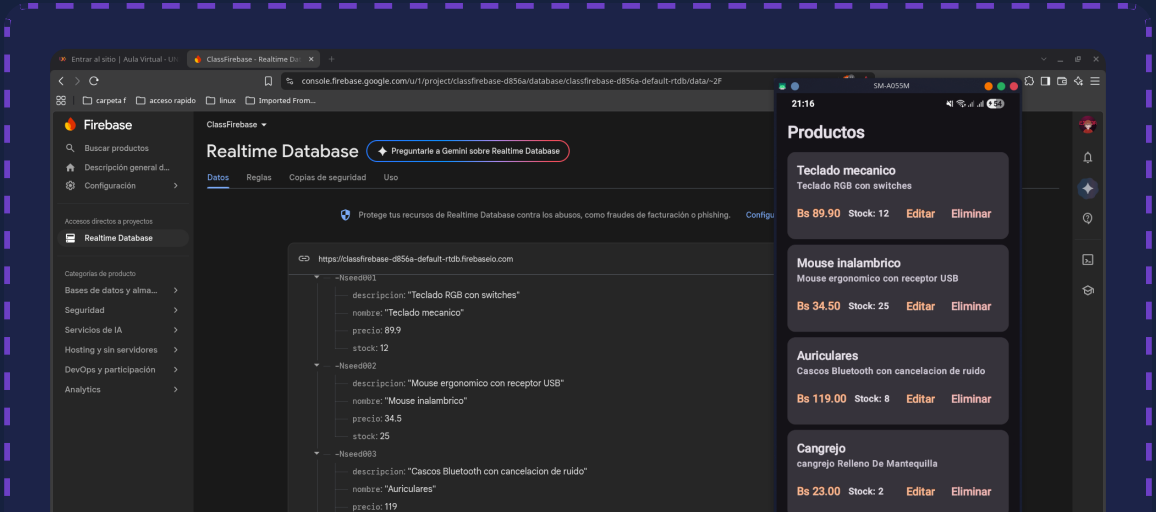
Listar productos



Crear y editar



Eliminar



Conclusión

Conectar Firebase a tu APK **no es magia**: es hacer peticiones HTTP a una URL, mandar y recibir JSON, y cuidar las reglas de seguridad.

Lo más importante: **diagnosticar mirando al servidor**, no suponer que el error está en la app.

¡Gracias!

Brayan Quispe Mamani

Universidad Franz Tamayo — 2026